

✓ 1. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")
```

✓ 2. Load Dataset

```
df = pd.read_csv('/content/TeachingRatings.csv')

print("Dataset shape:", df.shape)
display(df.head())
```

Dataset shape: (463, 13)

	rownames	minority	age	gender	credits	beauty	eval	division	native	tenure	students	allst
0	1	yes	36	female	more	0.289916	4.3	upper	yes	yes	24	
1	2	no	59	male	more	-0.737732	4.5	upper	yes	yes	17	
2	3	no	51	male	more	-0.571984	3.7	upper	yes	yes	55	
3	4	no	40	female	more	-0.677963	4.3	upper	yes	yes	40	
4	5	no	31	female	more	1.509794	4.4	upper	yes	yes	42	

✓ Q1. Duplicate Cases Using `prof`

`prof` identifies the instructor. Because an instructor may teach multiple courses, the same `prof` can occur in multiple rows.

`keep=False` marks **all rows** belonging to repeated professors as duplicates.

```
duplicates = df[df.duplicated(subset=['prof'], keep=False)].sort_values('prof')

print("All duplicate professor cases:")
display(duplicates)

print("Number of unique professors:", df['prof'].nunique())
```

All duplicate professor cases:

	rownames	minority	age	gender	credits	beauty	eval	division	native	tenure	students	all
0	1	yes	36	female	more	0.289916	4.3	upper	yes	yes	24	
94	95	yes	36	female	more	0.289916	3.7	upper	yes	yes	86	
95	96	yes	36	female	more	0.289916	3.6	upper	yes	yes	76	
96	97	yes	36	female	more	0.289916	4.4	upper	yes	yes	77	
97	98	no	59	male	more	-0.737732	4.0	upper	yes	yes	35	
...	...	...	...	...	...	...	...	...	...	...	...	
455	456	no	32	male	more	1.231394	4.2	lower	yes	yes	111	
93	94	yes	42	female	more	0.420400	3.3	upper	no	yes	48	
461	462	yes	42	female	more	0.420400	3.2	upper	no	yes	54	
460	461	yes	42	female	more	0.420400	3.3	upper	no	yes	52	
462	463	yes	42	female	single	0.420400	4.1	lower	no	yes	28	

456 rows × 13 columns

Number of unique professors: 94

Next steps: [Generate code with duplicates](#) [New interactive sheet](#)

Q1(a). Mean and Standard Deviation of Age — All Observations

```
mean_age_all = df['age'].mean()
std_age_all = df['age'].std()

print(f"Mean age (all observations): {mean_age_all:.4f}")
print(f"Standard deviation (all observations): {std_age_all:.4f}")
```

Mean age (all observations): 48.3650
Standard deviation (all observations): 9.8027

Q1(b). Keep One Observation Per Instructor

```
df_unique_prof = df.drop_duplicates(subset=['prof']).copy()

print("Observations after keeping one row per professor:", len(df_unique_prof))
print("Unique professors:", df_unique_prof['prof'].nunique())
```

Observations after keeping one row per professor: 94
Unique professors: 94

```
mean_age_unique = df_unique_prof['age'].mean()
std_age_unique = df_unique_prof['age'].std()

print(f"Mean age (one observation per professor): {mean_age_unique:.4f}")
print(f"Standard deviation (one observation per professor): {std_age_unique:.4f}")
```

Mean age (one observation per professor): 47.5532
Standard deviation (one observation per professor): 10.2565

Q1 Interpretation

For the standard TeachingRatings dataset, the expected results are approximately:

- All 463 observations: mean age **48.37**, SD **9.80**
- One observation per professor (94 observations): mean age **47.55**, SD **10.26**

The difference occurs because the original data contain course-level observations, so instructors with more courses can appear more often.

✓ Q2. Average Teaching Evaluation by Course Division

We compare the mean `eval` score for `lower` and `upper` division courses. A bar chart is suitable because `division` is categorical.

```
division_eval = df.groupby('division')['eval'].mean()

print("Average teaching evaluation by division:")
display(division_eval.round(4))
```

Average teaching evaluation by division:

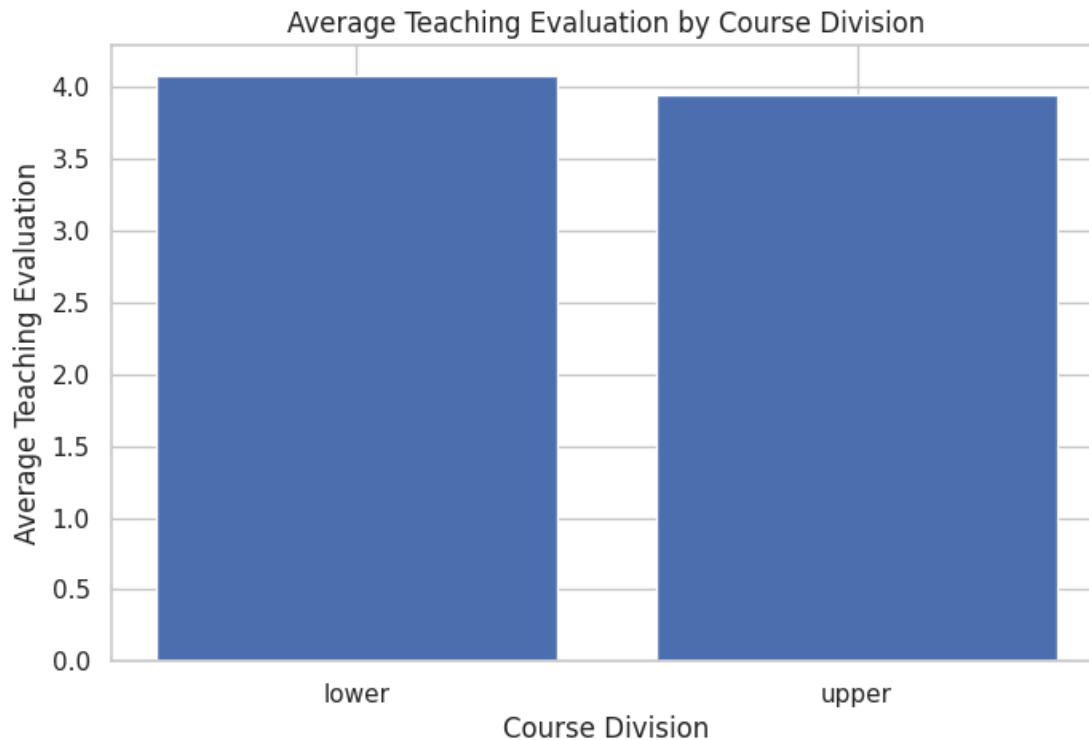
	eval
division	
lower	4.0873
upper	3.9526

dtype: float64

```
plt.figure(figsize=(8, 5))
plt.bar(division_eval.index, division_eval.values)

plt.xlabel('Course Division')
plt.ylabel('Average Teaching Evaluation')
plt.title('Average Teaching Evaluation by Course Division')

plt.show()
```



Q2 Interpretation

For the standard dataset:

- Lower division ≈ 4.09
- Upper division ≈ 3.95

Therefore, lower-division courses have the higher average teaching evaluation descriptively. This graph alone does not prove causation or statistical significance.

✓ Q3. Relationship Between Age and Teaching Evaluation

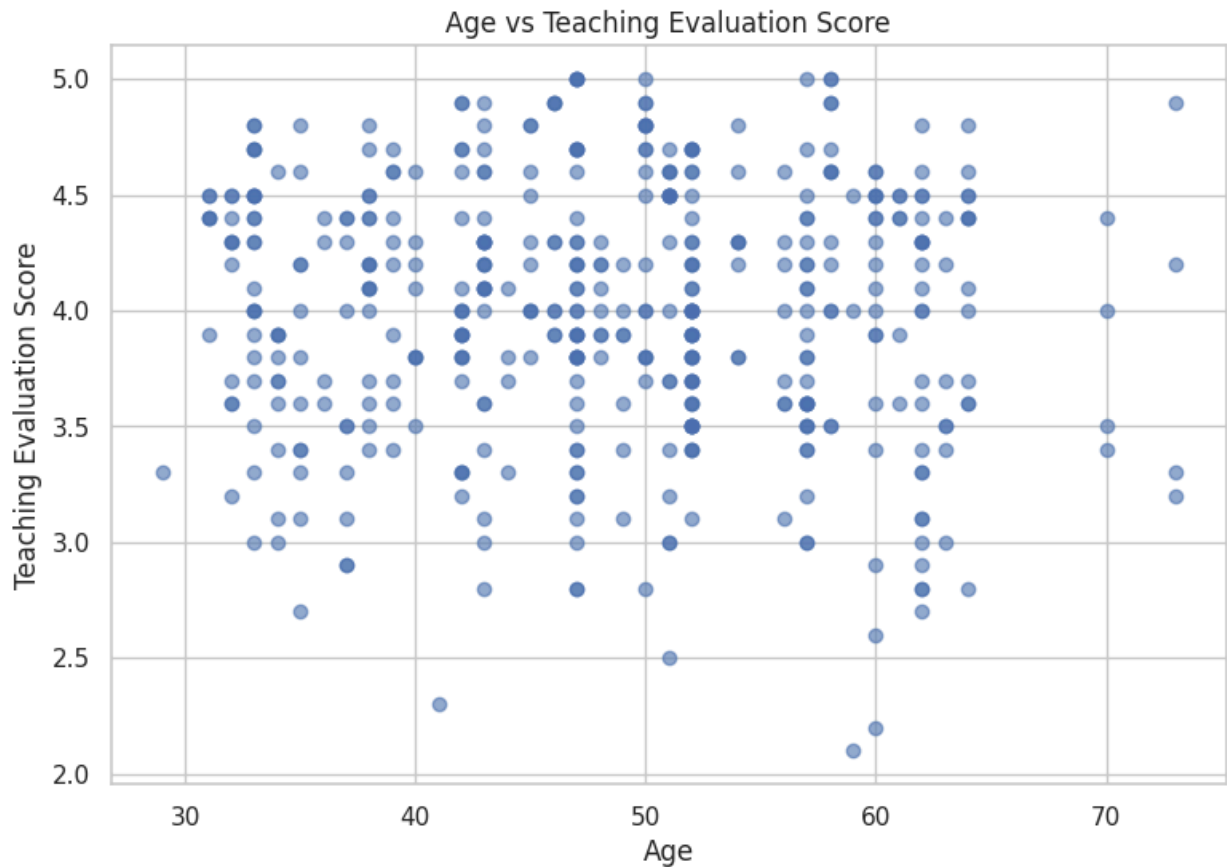
Both `age` and `eval` are numerical variables, so a scatterplot is appropriate. Each point represents an observation.

```
plt.figure(figsize=(9, 6))

plt.scatter(df['age'], df['eval'], alpha=0.6)

plt.xlabel('Age')
plt.ylabel('Teaching Evaluation Score')
plt.title('Age vs Teaching Evaluation Score')

plt.show()
```



✓ Q4. Age vs Teaching Evaluation — Differentiated by Gender

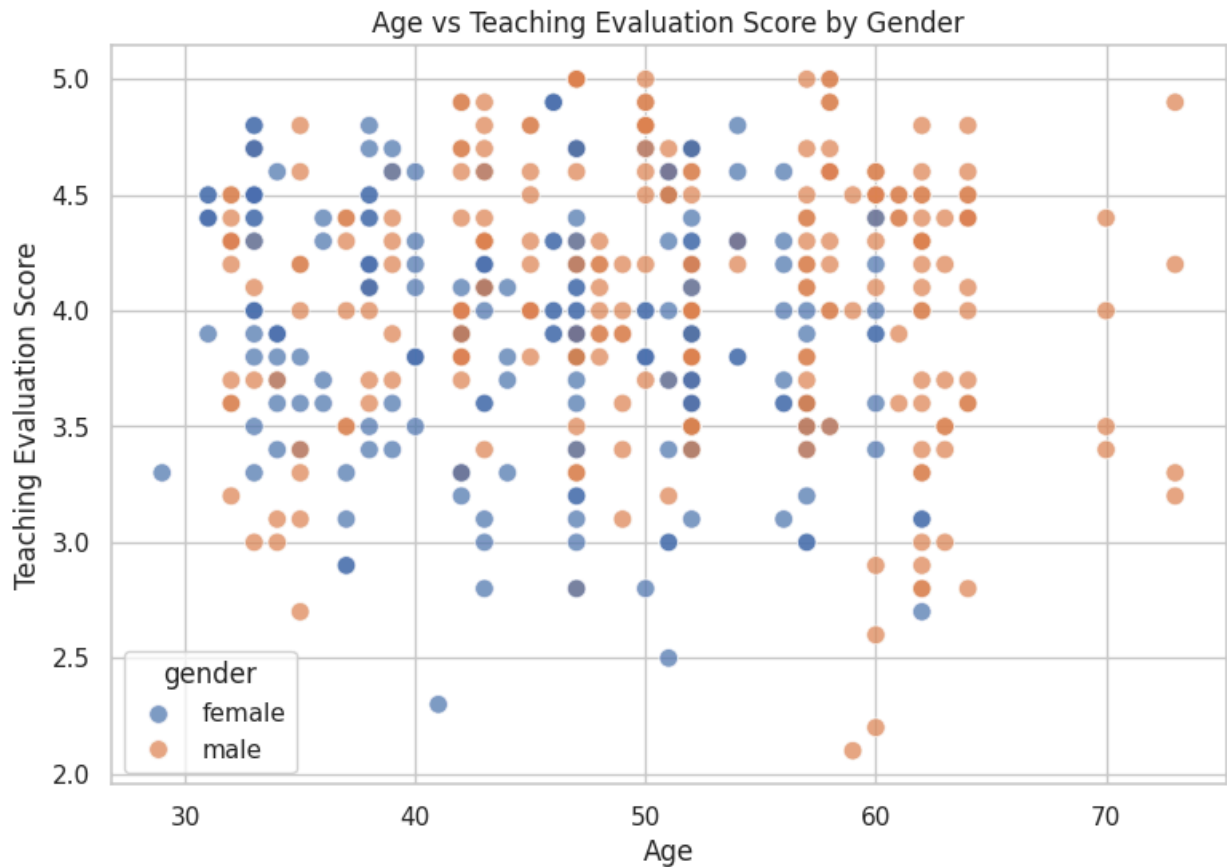
`hue='gender'` separates the observations into male and female groups.

```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x='age',
    y='eval',
    hue='gender',
    s=70,
    alpha=0.7
)

plt.xlabel('Age')
plt.ylabel('Teaching Evaluation Score')
plt.title('Age vs Teaching Evaluation Score by Gender')

plt.show()
```



Q5. Age vs Evaluation — Differentiated by Gender and Tenure

- `hue='gender'` differentiates male/female.
- `style='tenure'` differentiates tenured/non-tenured instructors by marker style.

```
plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df,
    x='age',
    y='eval',
    hue='gender',
    style='tenure',
    s=80,
    alpha=0.75
)

plt.xlabel('Age')
plt.ylabel('Teaching Evaluation Score')
plt.title('Age vs Teaching Evaluation by Gender and Tenure')

plt.show()
```

